USB-экран MK61s mini

Версия документа: 03.11.2026

Статус: реализовано в firmware и desktop-клиенте; wire protocol version 2.

Документ описывает решение целиком: пользовательский сценарий, устройство firmware, формат протокола, desktop-приложение, сборку обеих частей, бюджет памяти, диагностику и тестирование. Он рассчитан и на пользователя готовой сборки, и на разработчика, которому нужно изменить протокол или клиент.

Термины в документе:

- физический экран - установленный в устройстве LCD1602 A00/A02, OLED1602 WS0010 или UC1609;
- виртуальный экран - активная в USB-режиме графическая поверхность 192x64;
- устройство или firmware - MK61s mini co STM32F411;
- host или клиент - desktop-приложение на компьютере;
- foreground-интерфейс - текущий владелец экрана: калькулятор, меню, редактор FOCAL/TinyBASIC, проводник, WBMP, CHIP-8, диалог или другая модальная страница;
- CDC - существующий USB-последовательный интерфейс прошивки.

Цель

Добавить в прошивку режим USB-экран и поставляемое вместе с ним настольное приложение. После подключения MK61s mini к компьютеру приложение само включает режим, и устройство использует его окно как единственный активный графический дисплей. Выбор пункта меню на устройстве остаётся ручным резервным способом запуска.

Это не зеркалирование физического экрана. В USB-режиме активный экран всегда графический и многострочный, независимо от LCD1602 A00/A02, OLED1602 WS0010 или UC1609 в самом устройстве.

Для символьного LCD/OLED это именно смена display backend во время работы: после `ATTACH rows()` становится равным 6, редакторы и меню используют многострочные ветки, а аппаратно-зависимые A00/A02/WS0010-коды преобразуются в канонические графические токены. Русский текст передаётся как Unicode. Лимита HD44780 в восемь CGRAM знакомест на виртуальном экране нет. API пользовательских bitmap-глифов остаётся только для намеренно создаваемых во время работы картинок и не используется как таблица символов графического экрана.

Режим не превращает устройство в стандартный USB DisplayLink-монитор и не создаёт второй экран операционной системы. Изображение видно в отдельном приложении MK61 USB Screen. Другой serial terminal в это время открывать нельзя: экран, терминал приложения и ввод используют один CDC-порт.

Обязательное поведение

1. После открытия CDC-порта desktop-клиент сам запускает режим терминальной командой `uscreen`. Пункт Разработка → USB-экран (Development → USB Screen) остаётся ручной альтернативой.
2. До handshake с приложением физический экран может показывать ожидание.
3. После handshake физический экран гаснет и не получает обновлений до выхода из режима.
4. Приложение показывает весь обычный интерфейс: калькулятор, меню, редакторы, языковые среды, курсор, оверлеи, шрифты, WBMP, Markdown, CHIP-8 и анимацию.
5. Режим не блокирует основной цикл. Физическая клавиатура продолжает работать, а блокирующие меню, редакторы и ожидания клавиши обязаны вызывать фоновое обслуживание USB не реже heartbeat timeout.

6 При отключении USB, тайм-ауте heartbeat, остановке клиента или аварийном выходе физический экран включается с текущим foreground-интерфейсом. Редактор, меню или диалог не должны подменяться промежуточным экраном калькулятора. Графические fullscreen-модули WBMP и CHIP-8 являются исключением: они не могут продолжить работу на LCD1602, завершаются и показывают ГРАФИКА / НЕДОСТУПНА уже на физическом экране. Markdown вместо завершения переключается на plain text.

7 Долгое удержание ESC — аварийный выход; короткое нажатие остаётся доступным интерфейсу.

8 Интерактивный терминал работает одновременно с видеопотоком через тот же USB CDC и доступен прямо в desktop-клиенте.

9 В LCD1602-сборке настройки графического шрифта входят в прошивку при `MK61_ENABLE_USB_SCREEN=1`, но пункт меню виден только пока текущий экран является графическим USB-экраном.

Быстрый запуск готового решения

Подготовка firmware

USB Screen выключен в обычной сборке. В корне проекта запустите конфигуратор:

```
./tools/mk61-firmware.cmd
```

Выберите свою платформу и экран, затем откройте «Ключи компиляции», включите USB-экран и выполните «Собрать и прошить». Инструмент передаст компилятору `-DMK61_ENABLE_USB_SCREEN=1`. На Windows запускается тот же `tools\mk61-firmware.cmd`.

После прошивки пункт появляется в меню Разработка -> USB-экран (`Development -> USB Screen`). Если пункта нет, прошивка собрана с выключенной опцией.

Запуск desktop-приложения из исходников

```
cd apps/mk61_usb_screen
flutter pub get
flutter run -d macos
```

Для Windows и Linux замените цель на `windows` или `linux`. Полная подготовка среды, release-сборка и упаковка описаны в разделе «Сборка desktop-приложения».

Подключение

1 Подключите MK61s к компьютеру обычным USB-кабелем с линиями данных.

2 Запустите приложение и оставьте «Автоподключение» включённым либо выберите CDC-порт вручную.

3 Клиент откроет порт и отправит `uscreen`; firmware начнёт handshake из текущего интерфейса без навигации по меню. Терминальный провод остаётся CP1251, а клиент сам преобразует его в локальный Unicode.

4 Дождитесь статуса «USB Screen подключён». Только после подтверждения CAPS физический экран погаснет, а интерфейс станет графическим и многострочным.

Автоматический вход доступен из калькулятора, меню, проводника, FOCAL, TinyBASIC и других foreground-интерфейсов, которые обслуживают общий idle-loop. Если он не нужен, режим по-прежнему можно включить вручную через Разработка -> USB-экран.

Окно можно масштабировать; пиксели выводятся без сглаживания. Когда шторка терминала скрыта, обычная клавиатура компьютера управляет MK61. Когда терминал открыт, ввод кодируется клиентом в CP1251 и отправляется терминалу.

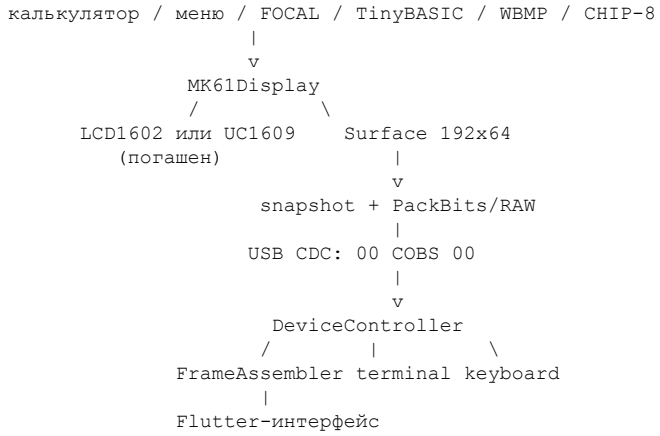
Выход и аварийное восстановление

- Кнопка «Отключить» или штатное закрытие приложения отправляет DETACH.
- При выходе из режима на самом MK61 firmware сначала отправляет встречный DETACH; приложение немедленно закрывает экранную сессию без сообщения об ошибке, оставляет CDC открытым и намеренно не включает режим снова.

- После такого выхода приложение показывает кнопку «Включить USB Screen». Можно нажать её либо запустить режим через меню устройства; новый `OFFER` будет принят на уже открытом порту.
- При аварийном завершении клиента firmware сама восстановит физический экран не позднее чем через 3000 мс после последнего корректного пакета.
- Долгое физическое удержание `ESC` в течение 1500 мс завершает USB Screen независимо от состояния компьютера.
- После выхода должен появиться текущий foreground-интерфейс. Если USB Screen был открыт из FOCAL, физический экран сразу показывает FOCAL, а не калькулятор.

Архитектура решения

Все экранные подсистемы firmware продолжают работать через существующий `MK61Display`. При handshake меняется его backend, а не код калькулятора, редакторов или меню:



Решение состоит из пяти независимых слоёв:

Слой	Ответственность
Display backend	Единая поверхность, текст, шрифты, курсор, WBMP, CHIP-8 и переключение физического экрана
Session	Handshake, heartbeat, снимок кадра, передача и виртуальные клавиши
Wire protocol	COBS, CRC16, типы пакетов, PackBits и terminal multiplex
Desktop controller	Поиск порта, handshake, проверка sequence/CRC, heartbeat и восстановление
Flutter UI	Пиксельный экран, физическая раскладка клавиш, терминал и диагностика

USB Screen компилируется как обычная возможность firmware, а не как отдельный fork для A00, A02, WS0010 или UC1609. Тот же wire protocol обслуживает все аппаратные варианты и сообщает клиенту только раскладку клавиатуры и профиль текста.

Виртуальный дисплей

Базовая поверхность — монохромный framebuffer 192x64. Текстовый интерфейс сохраняет 16 столбцов и существующие графические профили:

Шрифт	Строк
5×8	6
3×5	10

Протокол передаёт готовые пиксели. Клиент не дублирует растеризацию шрифтов и не интерпретирует аппаратно-зависимые экранные коды.

Настройки графического шрифта

Блок этих настроек компилируется, если физический экран — UC1609 или включён `MK61_ENABLE_USB_SCREEN`. Поэтому символьный A00/A02/WS0010 получает их в сборке с USB-экраном, но не в обычной LCD-сборке.

На LCD1602 пункт Настройки → Шрифт появляется динамически после handshake, когда текущей поверхностью уже стал USB-экран, и скрывается после возврата к физическому LCD. Доступны пресеты 5×8 и 3×5; выбор сразу применяется к виртуальной поверхности и сохраняется для следующих USB-сессий. Ключ `MK61_ENABLE_EXTENDED_FONT_SETTINGS=1` добавляет ручную настройку числа строк, высоты, межстрочного интервала и ширины глифа.

Сборка firmware

Поддерживаемые профили

USB Screen поддерживается одной реализацией во всех профилях сборщика:

ID профиля	Плата и дисплей
`mini-v3-a00`	mini V3, LCD1602 CGROM A00
`mini-v3-a02`	mini V3, LCD1602 CGROM A02
`mini-v3-ws0010`	mini V3, OLED1602 WS0010, English/Russian FT=10
`mini-v2-a00`	mini V2, LCD1602 CGROM A00
`mini-v2-a02`	mini V2, LCD1602 CGROM A02
`classic-v2`	Classic V2, UC1609
`classic-v3`	Classic V3, UC1609
`40th`	MK61s 40th, UC1609

Требуются `arduino-cli`, STM32 Arduino Core 2.12.0, LiquidCrystal 1.0.7 и STM32duino RTC 1.9.0. Проверить или установить закреплённые Arduino-зависимости можно через интерфейс сборщика либо командой:

```
./tools/mk61-firmware.cmd --setup
```

На Windows для загрузки также нужен STM32CubeProgrammer. На macOS/Linux сборщик использует `dfu-util` из STM32 Tools, Arduino, Homebrew/MacPorts или системного `PATH`.

Интерактивная сборка

1. Запустите `./tools/mk61-firmware.cmd`.
2. Выберите платформу и физический экран.
3. В «Ключах компиляции» включите USB-экран.
4. При необходимости включите расширенные настройки шрифтов. Обычные пресеты графического шрифта добавляются вместе с USB Screen автоматически.
5. Выберите «Только собрать .bin» либо «Собрать и прошить».

Выбор хранится в корневом `.mk61-firmware.conf`, который не попадает в Git. Проверить сохранённые параметры можно без сборки:

```
./tools/mk61-firmware.cmd --show-config
```

Критическая строка должна иметь вид:

```
MK61_ENABLE_USB_SCREEN=1
```

Этот флаг объявляет, что в прошивке существует графический backend, но не включает игровые или файловые модули сам по себе. CHIP-8 и отдельный WBMP-viewer без Markdown сохраняют собственные ключи:

```
MK61_ENABLE_WBMP_VIEWER=1
MK61_ENABLE_CHIP8=1
```

На LCD1602 WBMP=1 требует USB-экран только при выключенном Markdown; CHIP-8=1 требует его всегда. Модули тогда компилируются или создаются как System APP, но запуск возможен лишь после ATTACH, когда виртуальная поверхность действительно активна. CHIP-8 по умолчанию выключен. Полный контракт консоли C1 приведён в MK61s-mini-CHIP8.md.

После этого профиль можно собирать неинтерактивно. Пример для mini V3 A00:

```
./tools/mk61-firmware.cmd --profile mini-v3-a00 --build
```

Готовый комплект появляется в отдельном каталоге `.build/firmware/`, где `build.flags` содержит точный набор `-D`-ключей. Имя A00-артефакта: `.build/firmware/mk61s-M-mini-v3-lcd1602-a00-f411/mk61s-M-mini-v3-lcd1602-a00-f411.bin`.

Прямая сборка через arduino-cli

Для CI или диагностики можно вызвать компилятор без интерфейса. Каталог sketch должен называться `mk61s-M`; пример ниже предполагает, что содержимое `code/` уже скопировано в `/tmp/mk61s-M/`:

```
arduino-cli compile \
  --fqbn "STMicroelectronics:stm32:GenF4:pnum=BLACKPILL_F411CE,upload_method=dfuMethod,
xserial=none,usb=CDCgen,opt=osstd" \
  --build-property "compiler.cpp.extra_flags=-DMK61_LCD1602_A00 -DMK61_ENABLE_USB_SCREEN=1 -
DHAL_UART_MODULE_ONLY -DUSBD_CLASS_USER_STRING_DESC=0" \
  --build-property "compiler.c.extra_flags=-DHAL_UART_MODULE_ONLY -
DUSBD_CLASS_USER_STRING_DESC=0" \
  --build-property "compiler.c.elf.extra_flags=-Wl,--wrap=USBD_CDC_ClearBuffer,--
wrap=USBD_LL_SetupStage,--wrap=USBD_LL_Reset,--wrap=USBD_LL_Suspend,--wrap=USBD_LL_Resume,--
wrap=USBD_LL_DevConnected,--wrap=USBD_LL_DevDisconnected" \
  /tmp/mk61s-M
```

Последнее свойство обязательно для F401/F411 с USB CDC: оно включает NAK backpressure при заполнении входной очереди. `tools/mk61-firmware.cmd` и `tools/build_f401_bundle.sh` добавляют его автоматически.

Для A02 замените аппаратный ключ на `-DMK61_LCD1602_A02`, для OLED1602 — на `-DMK61_OLED1602_WS0010`; для остальных плат используйте точные флаги, которые показывает `./tools/mk61-firmware.cmd --list-profiles`. Не включайте одновременно два аппаратных профиля.

Выключение возможности

Значение по умолчанию в `code/config.h` равно нулю. При `MK61_ENABLE_USB_SCREEN=0` компилятор исключает session state, framebuffer, protocol service и пункт меню. Это важно для минимальных LCD1602-сборок: память не расходуется просто из-за наличия исходников в репозитории.

Транспорт и готовые клиенты

Выбран собственный двунаправленный протокол по уже существующему USB CDC.

RFB/VNC не выбран по следующим причинам:

- RFB 3.8 не поддерживает wire pixel format меньше 8 бит на пиксель. RAW-кадр был бы в восемь раз больше нативного однобитного кадра MK61s;
- обычные VNC-клиенты ожидают TCP, а noVNC — WebSocket. USB CDC не является ни тем, ни другим;
- USB-сеть потребовала бы новых USB descriptors, NCM или ECM/RNDIS для разных ОС, IP, DHCP и TCP-стека;
- serial-to-TCP/WebSocket bridge всё равно был бы собственным приложением, но с лишним сетевым слоем;
- Hextile/ZRLE/Tight заметно сложнее diff однобитных страниц и RLE для кадра всего 1536 байт.

Из RFB заимствуются полный или инкрементальный запрос, список прямоугольных обновлений и раздельные key-down/key-up.

Настольный клиент строится на Flutter и `flutter_libserialport`: один UI и один транспортный код для Windows, macOS и Linux, включая экранную клавиатуру и встроенный терминал.

Wire format 2

CDC рассматривается как один непрерывный поток с двумя логическими каналами:

- бинарный пакет экрана имеет вид `00 <COBS> 00`;
- ненулевые байты вне такой рамки — ввод или вывод интерактивного терминала.

Двойная граница нужна для однозначной повторной синхронизации после подключения посреди пакета или повреждения потока. Нулевой байт терминалу не требуется и в терминальном канале не передаётся. Декодированный бинарный пакет имеет layout:

Offset	Размер	Содержимое
0	2	magic, ASCII `MS` (`4D 53`)
2	1	версия, `02`
3	1	тип сообщения
4	1	flags; в текущих сообщениях равен нулю
5	2	sequence, little-endian
7	2	длина payload, little-endian, не больше 224
9	N	payload сообщения
9+N	2	CRC16-CCITT, little-endian

CRC вычисляется по байтам от magic до конца payload: начальное значение `0xFFFF`, polynomial `0x1021`, без финального XOR. Device и host ведут независимые 16-битные sequence и увеличивают их для каждого исходящего бинарного пакета; после `0xFFFF` счётчик переходит в ноль. Клиент отслеживает пропуски device sequence и запрашивает новый keyframe. Firmware не использует host sequence как подтверждение доставки.

Пакет `RECT` несёт одну страницу высотой 8 пикселей:

Поле	Байт	Описание
frame id	2	little-endian
x	1	0...191
page	1	0...7, `y = page * 8`
width	1	1...192
codec	1	`0` RAW, `1` PackBits
pixels	N	page-major LSB, декодируется ровно в `width` байт

Размер одного `RECT` ограничен одной страницей, поэтому encoder и CDC TX работают с фиксированным пакетом не больше 238 байт. Firmware хранит один неизменяемый snapshot текущей передачи размером 1536 байт: UI может продолжать менять рабочую поверхность, пока старый кадр уходит по USB. `FRAME_BEGIN` и `FRAME_END` делают восемь полос атомарными для клиента.

На UC1609 физический renderer обычно использует только первые 192 байта этого же массива как одностраничный буфер. После входа в USB Screen все 1536 байт становятся виртуальным framebuffer. То есть отдельные физический и USB framebuffer в этой сборке не выделяются. LCD1602 не имеет читаемой графической памяти, поэтому для него 1536-байтная поверхность выделяется отдельно.

Типы сообщений и payload

Type	Направление	Payload
`OFFER 0x10`	device → host	возможности без текстового профиля, 10 байт
`ATTACH 0x11`	host → device	пусто
`CAPS 0x12`	device → host	возможности и активный профиль, 14 байт
`DETACH 0x13`	оба направления	пусто
`PING` / `PONG 0x14/0x15`	оба направления	16-битный nonce
`REQUEST_KEYFRAME 0x16`	host → device	пусто
`FRAME_BEGIN 0x20`	device → host	frame id, keyframe flag, число `RECT`
`RECT 0x21`	device → host	область и пиксели, формат выше
`FRAME_END 0x22`	device → host	frame id и CRC16 всего framebuffer
`KEY_EVENT 0x30`	host → device	scan code и `down` (`0`/`1`)
`RELEASE_ALL_KEYS 0x31`	host → device	пусто

Payload `OFFER`/`CAPS` начинается с `width:u16`, `height:u8`, `page_height:u8`, `capabilities:u16`, `heartbeat_timeout_ms:u16`, `keyboard_layout:u8` и `key_count:u8`. `CAPS` дополнительно содержит `text_rows`, `glyph_width`, `glyph_height`, `line_gap`. Capability mask version 2 сообщает PackBits, key events, heartbeat, атомарные кадры, terminal multiplex и terminal `kbd`. Бит diff сохранён в формате для совместимости, но текущая RAM-экономная firmware его не объявляет.

Handshake

1. Сразу после открытия CDC-порта клиент отправляет обычную текстовую команду `uscreen\r` вне бинарной COBS-рамки. Терминал firmware выполняет её в общем idle-loop текущего foreground-интерфейса.
2. Команда `uscreen` (либо ручной пункт меню) переводит firmware в `WAITING_FOR_HOST`; firmware отправляет `OFFER` каждые 500 мс.
3. Клиент проверяет геометрию и версию, затем отправляет пустой `ATTACH`.
4. Firmware создаёт виртуальную поверхность, гасит физический экран и отвечает `CAPS` с активным текстовым профилем.
5. Клиент принимает режим только при геометрии 192x64, странице 8 пикселей и capability `ATOMIC_FRAMES`, затем отправляет `REQUEST_KEYFRAME`.
6. Firmware передаёт `FRAME_BEGIN`, восемь `RECT` и `FRAME_END`.
7. Клиент публикует изображение только после проверки количества областей и CRC всего framebuffer.

Команда идемпотентна: если ручной пункт меню уже перевёл firmware в ожидание, повторный `uscreen` от клиента не сбрасывает сессию.

При повторном `OFFER` на уже открытом порту клиент считает прежнюю сессию сброшенной, очищает зажатые клавиши и повторяет handshake без перезапуска приложения.

При штатном выходе, инициированном устройством, firmware отправляет пустой `DETACH` перед возвратом в `IDLE`. Клиент сразу снимает виртуальные удержания, сбрасывает кадр, но сохраняет открытый CDC-порт. Он подавляет автоматическую команду `uscreen`, иначе долгое физическое удержание `ESC` немедленно включало бы режим обратно. Следующий ручной `OFFER` принимается сразу; для явного повторного запуска в приложении есть кнопка «Включить USB Screen». Физическое переподключение кабеля считается новой сессией и снова разрешает автозапуск. Отправка `DETACH` ограничена по времени; при обрыве кабеля резервным механизмом остаётся heartbeat timeout.

Payload возможностей

Offset	Поле OFFER/CAPS	Значение текущей firmware
0	`width:u16`	192
2	`height:u8`	64
3	`page_height:u8`	8
4	`capabilities:u16`	маска возможностей
6	`heartbeat_timeout_ms:u16`	3000
8	`keyboard_layout:u8`	0 Mini, 1 Classic, 2 40th
9	`key_count:u8`	40
10	`text_rows:u8`	только CAPS
11	`glyph_width:u8`	только CAPS
12	`glyph_height:u8`	только CAPS
13	`line_gap:u8`	только CAPS

OFFER имеет длину 10 байт, CAPS - 14 байт.

Бит	Имя	Текущий статус
0	`DIFF`	определён, но не объявляется
1	`PACKBITS`	объявляется
2	`KEY_EVENTS`	объявляется
3	`HEARTBEAT`	объявляется
4	`ATOMIC_FRAMES`	объявляется и обязателен для клиента
5	`TERMINAL_MUX`	объявляется
6	`TERMINAL_KBD`	объявляется; доступны команды `kbd XX`

Payload кадров и управления

`FRAME_BEGIN` содержит `frame_id:u16`, `keyframe:u8` и `rect_count:u8`. Текущая firmware всегда ставит `keyframe=1` и `rect_count=8`. `FRAME_END` содержит тот же `frame_id:u16` и CRC16 полного 1536-байтного framebuffer. `PING` и `PONG` переносят один 16-битный nonce без изменения.

`KEY_EVENT` содержит `scan_code:u8` и `down:u8`; допустимы `down=0` и `down=1`. Пустой `RELEASE_ALL_KEYS` снимает все удержания host-клавиш. Эти бинарные сообщения используются для экранной клавиатуры с настоящими down/up; короткие последовательности клавиатуры ПК обычно идут командами `kbd XX` по текстовому каналу, чтобы повторные SMS-tap не сливались.

Кадры и сжатие

Полный кадр занимает 1536 байт. При каждой новой ревизии firmware снимает один snapshot и передаёт полный keyframe из восьми страниц. Если интерфейс успел измениться ещё раз во время передачи, промежуточные ревизии объединяются и следом уходит только последнее состояние. Полный keyframe также передаётся:

- при входе в режим;
- после переподключения;
- по запросу клиента;
- после нарушения sequence или явного запроса восстановления.

Такой режим убирает из STM32 предыдущий framebuffer и таблицу diff — экономится около 1,36 КиБ RAM даже после добавления очереди терминала. Цена — до 1536 байт несжатых пикселей на изменение, что мало для native USB CDC Full Speed.

Бюджет RAM и flash

Точные числа зависят от контроллера, профиля и набора System APP и должны сниматься из текущего `arduino-cli` build report. Нельзя сравнивать сборки с разными значениями WBMP или CHIP-8: после перехода на общее графическое правило оба модуля отсутствуют в обычной LCD1602-сборке без USB Screen.

Неизменяемые архитектурные требования:

- USB Screen содержит полный кадр 1536 байт, snapshot протокола, parser, terminal FIFO и очередь виртуальных клавиш;
- вход в режим не вызывает `malloc` и не создаёт новый runtime-буфер;
- на UC1609 рабочая видеопамять совмещается с существующим `render_buffer`;
- WBMP использует общий `language_workspace`, а не постоянный второй кадр;
- Markdown хранит event stream и текущий кадр в том же `language_workspace`;
- CHIP-8 размещает состояние VM и выходной кадр в том же `language_workspace`;
- на F401 реализации WBMP, Markdown и CHIP-8 загружаются по одной сверху свободной RAM. Каждая занимает свой размер образа/BSS с округлением до 32 байт; постоянного 20-КиБ APP-резерва нет.

Размеры сборок с каждым сочетанием ключей фиксируются автоматическими size-regression тестами. Измерение для документа необходимо обновлять после полной сборки финального набора, а не переносить из прежней конфигурации с LCD1602/CGRAM viewer.

USB-экран выключен по умолчанию и является отдельным ключом сборки. В `tools/mk61-firmware.cmd` он переключается в пункте «Ключи компиляции» и сохраняется в `.mk61-firmware.conf` как `MK61_ENABLE_USB_SCREEN=0` или `1`. Для использования режима нужно выбрать значение `1`. Значение `0` исключает из прошивки сессию протокола, виртуальную поверхность и пункт Разработка → USB-экран. При этом отдельный WBMP viewer (`MARKDOWN=0, WBMP=1`) или `MK61_ENABLE_CHIP8=1` для LCD1602 становится недопустимой комбинацией.

PackBits и RAW fallback

Каждая область имеет тип кодека. Сжатие используется только если результат меньше RAW. Для wire version 2 выбран PackBits/RLE с RAW fallback:

- кодер не требует словаря и динамической памяти;
- одинаковые байты фона и заливок сжимаются до двух байт;
- на несжимаемых данных перерасход ограничен, но RAW fallback не даёт ему попасть в wire;
- codec id оставляет место для XOR-RLE или LZSS в следующей версии.

PackBits декодируется до `width` байт области:

- control `0...127` означает `control + 1` следующих literal-байтов;
- control `129...255` означает `257 - control` повторов следующего байта;
- control `128` отклоняется как неканонический no-op;
- выход меньше или больше ожидаемой ширины делает весь кадр ошибочным.

Кодер выбирает PackBits только при строго меньшем размере. В противном случае область передаётся RAW, поэтому несжимаемое изображение не получает RLE-overhead.

LZSS не используется: для кадра в 1536 байт его словарь, кодер и CPU cost не оправданы без измеренного выигрыша над постраничным PackBits/RLE.

Скорость CDC

Сборка использует нативный `USBSerial` STM32 (`usb=CDCgen`), не физический UART. В STM32 Arduino core параметр `USBSerial::begin(baud)` намеренно игнорируется; `115200` в firmware и приложении является совместимым line-coding значением, а не ограничением потока. Фактическая передача использует 64-байтный USB FS endpoint и 128-байтную очередь core. Sender заполняет доступное место через `availableForWrite()` и не содержит искусственного ограничения fps.

`usb_screen::service()` рассчитан на частый неблокирующий вызов:

- за один вызов принимается не больше 96 входных байтов;
- передача продолжается только на объём, который сообщает `Serial.availableForWrite()`;
- один wire-пакет не превышает 238 байт;
- UI меняет рабочую поверхность независимо от неизменяемого TX-snapshot;
- если за время передачи произошло несколько перерисовок, они объединяются в один следующий кадр последней ревизии.

Пока бинарный пакет частично находится в TX, сервисы, способные печатать в Serial, временно не выводят текст. Калькулятор, клавиатура, звук и экранные вычисления при этом продолжают работать. Это сохраняет границы COBS без длинной блокировки основного цикла.

Одновременный терминал

Firmware разбирает входной CDC-поток до передачи его бинарному parser:

- байты внутри `00 ... 00` идут state machine USB Screen;
- обычные байты вне рамки попадают в 256-байтную очередь интерактивного терминала;
- недописанный бинарный пакет всегда завершается до любого `Serial.print`, поэтому эхо или длинный ответ команды не могут повредить видеокадр;
- фоновые `.m61`-скрипты и терминал выполняются только между полными бинарными пакетами, но продолжают работать в активном USB Screen.

Вне USB Screen обычный serial terminal теперь также обслуживается из `idle_main_process()`, а не только из верхнего `loop()`. Поэтому стартовая команда `uscreen` принимается, даже если ввод ждёт блокирующий редактор FOCAL, TinyBASIC, меню или другой foreground-интерфейс. В `.m61`-сценариях команда запрещена явным allowlist, чтобы файл данных не мог самовольно выключить физический экран.

Desktop-клиент демультиплексирует тот же поток, показывает до 64 КиБ вывода, обрабатывает CR/LF/backspace/tab как терминал, декодирует проводной CP1251 в Unicode и кодирует команды обратно в CP1251 с `CR`. Прошивка не переключает кодировку. M61-команда `print` в CDC-терминал не попадает: прошивка сама применяет её текст и ANSI-команды к активному экранному буферу, после чего desktop-клиент получает уже готовый кадр. ANSI, пришедший обычным терминальным выводом, клиент по-прежнему разбирает отдельно; цвет и прочие атрибуты SGR не рисуются. Одна строка ограничена 238 байтами после CP1251-кодирования. Символ, которого нет в CP1251, отклоняется клиентом. Нулевые байты и встроенные CR/LF из строки удаляются, чтобы пользовательская команда не могла создать бинарную рамку или несколько команд.

CDC-порт должен принадлежать одному процессу. Пока MK61 USB Screen подключён, нельзя одновременно открывать его Arduino Serial Monitor, `screen`, `minicom` или другим терминалом. Все обычные команды (`help`, `ver`, `reg`, `ls`, `dfu` и остальные) выполняются во встроенной шторке приложения.

Источники архитектурного решения

- RFB Protocol 3.8
- noVNC: server requirements
- TinyUSB network device example
- flutter_libserialport

Виртуальная клавиатура

Desktop-клиент строит матрицу по `keyboard_layout` (`Mini`, `Classic`, `40th`). Mini показывается в физическом порядке 5x8 реальной клавиатуры, включая основные подписи и дополнительные обозначения слоёв F, K, a...e. Подпись не определяет wire-код: каждая кнопка хранит действие, которое преобразуется в scan-code именно выбранной firmware-раскладки.

Есть два пути ввода:

1. Mouse/touch на экранной клавиатуре отправляет бинарные KEY_EVENT `down/up`. Поэтому работают удержание, несколько одновременно нажатых кнопок и отпускание в правильном порядке.
2. Прямые клавиши ПК также отправляют бинарные KEY_EVENT `down/up`. Хост выбирает scan-code матрицы из активной раскладки, а текущий foreground firmware сам трактует нажатие, отпускание и удержание. Только буквы и знаки, требующие K/F и многонажатийного ввода, передаются последовательностями `kbd XX` через terminal multiplex.

Пока терминальная шторка скрыта, приложение читает физические клавиши как фиксированную US-QWERTY. Активная русская или другая раскладка macOS, Windows или Linux не меняет ввод в MK61, и приложение не переключает системный язык. `Shift` выбирает американский символ второго регистра; `Command`, `Control` и `Option/Alt` не превращаются в кнопки калькулятора.

Поддерживаются:

- A...Z, преобразуемые в прописные буквы через штатный K + многонажатийный алфавит;
- цифры основной клавиатуры и numpad;
- +, -, *, /, ^, точка, пробел и знаки `! @ # $ % & ( ) : ; , " = ' < >`;
- стрелки, Backspace/Delete, Enter как 0K и Escape как ESC; на Mini ↑ использует ←ШГ (`kbd 20`), ↓ — ШГ→ (`kbd 21`), а ←/→ остаются отдельными стрелками;
- F1 как K, F2 как F, F3 как USER, F5 как С/П, F6 как SAVE, F7 как LOAD.

Неподдерживаемая печатная клавиша поглощается и ничего не отправляет. Когда шторка терминала открыта, правило US-QWERTY отключается: поле ввода получает обычный текст текущей системной раскладки и IME, кодирует его в CP1251 и отправляет терминалу. Непредставимые символы не заменяются молча, а показывают ошибку ввода. Пока шторка скрыта, приложение помечает все четыре события стрелок обработанными: они не перемещают фокус, не раскрывают элементы и не прокручивают интерфейс GUI.

При потере фокуса, отключении и закрытии клиент отправляет RELEASE_ALL_KEYS, чтобы в firmware не оставалось логически зажатой клавиши.

Firmware сводит физические и виртуальные события в обычную очередь клавиатуры, поэтому калькулятор, меню и языковые режимы не знают источник нажатия. Одновременно состояние внешних `down/up` доступно foreground-модулям для неблокирующего опроса удержаний. Фронт каждого `down` фиксируется отдельно: даже если короткий mouse/touch-клик успел отпуститься до следующего опроса, CHIP-8 видит нажатие хотя бы один цикл. Каждая строка `kbd XX` кладёт ровно один scan-code в очередь и в тот же фиксатор фронта; следующая строка разбирается на следующей idle-итерации. Это исключает потерю повторных SMS-tap, например вторых нажатий в `H = K,4,4` и `L = K,5,5`. Промежуточная очередь вмещает полный цикл всех 40 клавиш - 40 нажатий и 40 отпусканий. Она отдельно учитывает host-requested и уже доставленные калькулятору нажатия. При DETACH недоставленные события отбрасываются, а release создаётся только для фактически доставленного down, поэтому на физический foreground не может протечь «фантомная» клавиша из завершённой сессии. Состояние внешних клавиш и генерация hold/unhold вынесены в чистые модули и проверяются отдельно, включая несколько одновременно нажатых клавиш и переполнение 32-битного счётчика времени.

FOCAL и TinyBASIC имеют собственные foreground-циклы редакторов. Они явно вызывают `idle_main_process()` на каждой итерации, поэтому heartbeat, отправка кадров, terminal multiplex и `kbd` продолжают работать даже когда редактор долго ждёт ввода. Исполнение программ также обслуживает USB: калькулятор — на каждом кванте основного цикла, FOCAL — перед каждым оператором (включая тело FOR), TinyBASIC — на каждой исполняемой строке и итерации перехода.

Состояния и восстановление

Firmware использует состояния IDLE, WAITING_FOR_HOST, ATTACHED.

Состояние	Физический экран	Поведение USB
IDLE	включён	USB Screen не предлагает сессию
WAITING_FOR_HOST	включён	OFFER каждые 500 мс
ATTACHED	погашен	кадры, heartbeat, terminal и клавиатура

Переходы:

- команда uscreen от desktop-клиента или выбор пункта меню вызывает start() и переводит IDLE -> WAITING_FOR_HOST;
- корректный ATTACH переводит WAITING_FOR_HOST -> ATTACHED;
- DETACH или 3000 мс без корректного host-пакета переводят ATTACHED -> WAITING_FOR_HOST, включают физический экран и позволяют приложению переподключиться без повторного выбора меню;
- долгое физическое удержание ESC в течение 1500 мс вызывает cancel() и передаёт device → host DETACH, после чего переводит любое активное состояние в IDLE; клиент закрывает экранную сессию, сохраняет порт и не запускает её повторно без явного действия пользователя;
- короткий ESC отменяет первоначальное ожидание, пока foreground-цикл пункта меню ещё ждёт первый handshake. После уже состоявшейся сессии короткий ESC остаётся обычной клавишей текущего интерфейса.

Host отправляет PING каждые 750 мс. Любой корректный бинарный host-пакет обновляет deadline firmware; корректный PING получает PONG с тем же nonce. Клиент со своей стороны ждёт device-пакет не дольше объявленного timeout плюс 500 мс и при потере связи закрывает порт либо возвращается к автопоиску.

Переключение display backend

UC1609-сборка при входе переносит в Surface логическую text grid, пользовательские глифы, позицию и режим курсора, активный шрифт и top-right overlay. Поэтому первый keyframe уже содержит текущий modal foreground даже у экрана, который ждёт ввод через общий get_key_wait() и сам не следит за display revision. При выходе текстовое состояние копируется обратно в UC1609 backing grid и физическая панель включается только после её перерисовки.

При входе символьная LCD/OLED-сборка читает из собственного компактного shadow текущие 2x16 знакомест, CGRAM-глифы и курсор. Затем создаётся поверхность 192x64, первые две строки заполняются этим состоянием, rows() меняется на число строк выбранного графического профиля, увеличивается displayModeRevision, и только после этого физическая панель гасится. Следующая перерисовка foreground уже использует многострочную геометрию.

При выходе порядок обратный и специально защищён от вспышки калькулятора:

1. Firmware снимает первые две строки текущей виртуальной текстовой поверхности, её CGRAM-глифы и состояние курсора.
2. Виртуальная поверхность завершается, но LCD1602 пока остаётся погашенным.
3. Снятое состояние записывается в физический символьный backend.
4. Физический экран включается уже с текущим foreground-интерфейсом.
5. Revision дисплея заставляет активное меню или редактор дорисовать себя.

Верхний цикл вызывает lcd_std_display_redraw() только если foreground действительно принадлежит калькулятору. FOCAL, TinyBASIC и class_menu следят за той же revision внутри собственных циклов. Поэтому выход из USB Screen в редакторе FOCAL не должен требовать дополнительного ESC для восстановления правильного изображения.

Fullscreen-обработчики следят за `displayModeRevision` в собственном цикле. WBMP, Markdown и CHIP-8 могут перепривязать кадр после перехода между двумя графическими backend. При возврате с USB Screen на LCD1602 WBMP и CHIP-8 сначала освобождают fullscreen и завершают `FILE_OPEN` результатом `UNSUPPORTED_DISPLAY`. Проводник уже на восстановленном физическом экране показывает ГРАФИКА / НЕДОСТУПНА; скрытое исполнение ROM или просмотрщика не продолжается. Markdown освобождает framebuffer и перерисовывает ту же страницу как plain text.

Desktop-клиент

Исходники находятся в `apps/mk61_usb_screen`. Клиент для macOS, Windows и Linux:

- автоматически выбирает только известный USB VID/PID и product identity MK61/BLACKPILL_F411CE, отправляет команду `uscreen` и проверяет `OFFER`;
- проверяет COBS, version, length, packet CRC, sequence и framebuffer CRC;
- применяет области кадра только после корректного `FRAME_END`;
- запрашивает keyframe после потери sequence или повреждённого кадра;
- масштабирует монохромные пиксели без сглаживания и предлагает три палитры;
- показывает скорость, объём RX/TX, потери и ошибки parser;
- содержит ручной выбор порта, полноценную виртуальную клавиатуру и выдвижной терминал, работающий одновременно с экраном;
- пока терминал скрыт, направляет обычную клавиатуру ПК в интерфейс MK61; при открытии переводит фокус в неограниченную матрицей MK61 строку USB-терминала;
- поглощает стрелки на уровне корневого keyboard focus, чтобы после передачи в MK61 они не использовались навигацией или прокруткой Flutter-интерфейса;
- принимает device → host `DETACH` как штатное завершение, без показа ошибки, оставляет порт открытым и предлагает явное повторное включение.

Production-клиент использует `libserialport`, но контроллер зависит от узкого интерфейса serial-транспорта. В тестах к нему подключается детерминированное fake-устройство: через настоящий parser и state machine оно проводит `OFFER` → `ATTACH` → `CAPS`, terminal/video multiplex, фрагментированный keyframe, виртуальные клавиши, heartbeat, ошибки sequence/CRC и повторный handshake после сброса сессии. Отдельные regression-тесты проверяют allowlist serial-портов, capability gating, drain перед detach и изолированные каналы обновления display/terminal.

Команды запуска, release-сборки и права на serial-порт описаны в `apps/mk61_usb_screen/README.md`.

Состояния клиента

`DeviceController` проходит состояния `disconnected`, `scanning`, `opening`, `waitingForOffer`, `attaching`, `attached` и `error`. Автопоиск выполняется раз в секунду, но probe разрешён только для USB `0483:5740`, чей product/description содержит identity `MK61`, `MK-61` или `BLACKPILL_F411CE`. Имена `usbmodem`, `usbserial`, `ttyACM`, `ttyUSB` и `COM<number>` сами по себе не считаются идентификацией. Все остальные порты видны для явного ручного выбора, но автоматически не открываются и не получают текст `uscreen`.

Сразу после открытия порт получает `uscreen\r`, затем клиент ждёт `OFFER` 3500 мс, а после `ATTACH` ждёт `CAPS` ещё 1800 мс. Firmware вместо безусловной задержки старта до 1800 мс ждёт DTR: открытие CDC-порта завершает ожидание сразу, а уже принятый `uscreen` остаётся во входном буфере. Автоматический probe молча переходит к следующему кандидату; ручной выбор показывает ошибку пользователю. При закрытии приложение сначала снимает виртуальные удержания, отправляет `DETACH`, дожидается передачи через `drain()`, отменяет подписку на serial и только затем закрывает дескриптор. При встречном `DETACH` от firmware ответный пакет не посылается: клиент сбрасывает кадр, остаётся в `waitingForOffer` на том же дескрипторе и показывает кнопку повторного запуска. Это отличает осознанный выход на устройстве от физического отключения USB; после настоящего нового подключения автоматическая команда снова разрешена.

FrameAssembler имеет опубликованный и staging framebuffer. `FRAME_BEGIN` создаёт staging-копию либо чистую поверхность для keyframe, `RECT` меняют только staging, а `FRAME_END` проверяет frame id, число

областей и CRC. Ошибка отбрасывает staging целиком и вызывает REQUEST_KEYFRAME, поэтому пользователь никогда не видит половину нового кадра.

Сборка desktop-приложения

Исходники клиента находятся в apps/mk61_usb_screen. Это обычный Flutter desktop-проект без отдельного backend-сервера. Native-доступ к serial даёт пакет flutter_libserialport; его библиотеки должны поставляться вместе с готовым bundle.

Версии и общая подготовка

CI проекта использует Flutter 3.41.9 stable. pubspec.yaml требует Dart ^3.11.5; практический воспроизводимый вариант - установить ту же Flutter 3.41.9. Проверьте среду:

```
flutter --version
flutter doctor -v
flutter devices
```

В корне приложения получите зависимости:

```
cd apps/mk61_usb_screen
flutter pub get
```

pubspec.lock хранится в Git, поэтому не удаляйте его для обычной сборки. Команда flutter pub get должна использовать зафиксированные совместимые версии пакетов.

macOS

Нужны macOS, Xcode с Command Line Tools и CocoaPods. Один раз включите desktop, если цель ещё не видна:

```
flutter config --enable-macos-desktop
flutter doctor -v
```

Запуск debug-сборки:

```
cd apps/mk61_usb_screen
flutter run -d macos
```

Release-сборка:

```
flutter build macos --release
```

Результат:

```
apps/mk61_usb_screen/build/macos/Build/Products/Release/mk61_usb_screen.app
```

Минимальная версия системы в проекте - macOS 10.15. App Sandbox намеренно выключен, иначе приложение не сможет напрямую открыть /dev/cu.usbmodem*. Обычная локальная сборка не выполняет Developer ID notarization; для внешнего распространения подпись и notarization нужно добавить отдельно.

Архив, аналогичный CI, создаётся из каталога приложения:

```
ditto -c -k --sequesterRsrc --keepParent \
  build/macos/Build/Products/Release/mk61_usb_screen.app \
  MK61-USB-Screen-macOS.zip
```

Windows

Нужны Windows 10/11, Flutter stable и Visual Studio 2022 с workload «Desktop development with C++» и Windows SDK. Это именно Visual Studio, а не только Visual Studio Code. Включите desktop-цель и проверьте toolchain:

```
flutter config --enable-windows-desktop
flutter doctor -v
```

Debug-запуск и release-сборка:

```
cd apps\mk61_usb_screen
flutter pub get
flutter run -d windows
flutter build windows --release
```

Готовый каталог:

```
apps\mk61_usb_screen\build\windows\x64\runner\Release\
```

Распространять нужно весь каталог `Release`, а не один `.exe`: рядом лежат Flutter runtime, data и native-библиотека serialport. Упаковка:

```
Compress-Archive `
  -Path build\windows\x64\runner\Release\* `
  -DestinationPath MK61-USB-Screen-Windows-x64.zip
```

Стандартный USB CDC ACM обычно использует системный драйвер Windows.

Linux

Для Ubuntu/Debian установите тот же набор native-зависимостей, что использует CI:

```
sudo apt-get update
sudo apt-get install -y clang cmake ninja-build pkg-config \
  libgtk-3-dev libstdc++-12-dev
flutter config --enable-linux-desktop
flutter doctor -v
```

Debug-запуск и release-сборка:

```
cd apps/mk61_usb_screen
flutter pub get
flutter run -d linux
flutter build linux --release
```

Готовый переносимый bundle:

```
apps/mk61_usb_screen/build/linux/x64/release/bundle/
```

Запускайте `bundle/mk61_usb_screen`; не переносите бинарник отдельно от `lib/` и `data/`. Архив, аналогичный CI:

```
tar -C build/linux/x64/release \
  -czf MK61-USB-Screen-Linux-x64.tar.gz bundle
```

Пользователь должен иметь доступ к CDC-порту. На распространённых Debian/Ubuntu-системах это обычно группа `dialout`:

```
sudo usermod -aG dialout "$USER"
```

После изменения группы нужен новый вход в систему. На дистрибутивах с другим udev-правилом проверьте владельца конкретного `/dev/ttyACM*` или `/dev/ttyUSB*`; запуск всего GUI от root не рекомендуется.

Анализ, тесты и повторная чистая сборка

Перед release выполните:

```
cd apps/mk61_usb_screen
flutter analyze
flutter test
```

Если native build cache повреждён или после обновления Flutter остались старые артефакты:

```
flutter clean
flutter pub get
flutter build macos --release
```

Последнюю строку замените на целевую платформу. macOS, Windows и Linux нужно собирать нативно на соответствующей ОС; стандартный workflow не кросс-компилирует desktop targets.

CI и готовые архивы

Workflow `.github/workflows/usb-screen-desktop.yml` запускается при изменении приложения, на pull request и вручную. Сначала Linux job выполняет `flutter analyze` и `flutter test`, затем matrix нативно собирает:

- `MK61-USB-Screen-macOS.zip`;
- `MK61-USB-Screen-Windows-x64.zip`;
- `MK61-USB-Screen-Linux-x64.tar.gz`.

Архивы публикуются как GitHub Actions artifacts с retention 14 дней. Workflow не создаёт GitHub Release и не подписывает приложения: официальный релиз при необходимости должен отдельно скачать artifacts, подписать их и прикрепить к тегу.

Карта исходного кода

Firmware

Файл	Назначение
`code/config.h`	Compile-time флаг и значение по умолчанию
`code/usb_screen_protocol.hpp/.cpp`	Wire constants, COBS, CRC16, PackBits, parser и encoder
`code/usb_screen_surface.hpp/.cpp`	Графическая поверхность 192x64, текст, курсор, bitmap и overlay
`code/usb_screen.hpp/.cpp`	Session state, heartbeat, кадры, terminal FIFO и виртуальные клавиши
`code/display.hpp/.cpp`	Переключение backend, framebuffer и восстановление физического экрана
`code/menu.cpp`	Пункт ожидания клиента и redraw меню по display revision
`code/development.cpp`	Локализованный пункт `USB Screen`/USB-экран
`code/mk61s-M.ino`	Фоновый service, terminal multiplex и redraw владельца foreground
`code/focal.cpp`	Service и redraw внутри редактора/исполнителя FOCAL
`code/tinybasic.cpp`	Service и redraw внутри редактора/исполнителя TinyBASIC

Desktop

Файл	Назначение
`lib/protocol/mk61_protocol.dart`	Packet codec, stream parser, capabilities и FrameAssembler
`lib/device/serial_transport.dart`	Узкая обёртка над `flutter_libserialport` и fakeable interface
`lib/device/device_controller.dart`	Поиск порта, handshake, heartbeat, terminal и keyboard transport
`lib/device/keyboard_definition.dart`	Матрицы Mini, Classic, 40th и подписи физических клавиш
`lib/device/host_keyboard_mapping.dart`	Fixed US-QWERTY и последовательности для букв/знаков
`lib/ui/virtual_display.dart`	Пиксельный renderer без сглаживания
`lib/ui/virtual_keyboard.dart`	Responsive 5x8-клавиатура и down/up мыши/touch
`lib/ui/home_page.dart`	Порты, экран, focus routing, терминал и диагностика
`lib/main.dart`	Desktop window, тема и начальный размер

Тесты firmware находятся в `tests/usb_screen_*_self_test.cpp`, а тесты клиента

- в `apps/mk61_usb_screen/test/`. CI клиента описан в `.github/workflows/usb-screen-desktop.yml`; compile-check firmware - в `.github/workflows/firmware-release.yml`.

Диагностика и типовые проблемы

В приложении раскройте «Диагностика соединения». Счётчики кадров, fps, принятых байтов, sequence gaps, отброшенных пакетов и кадров помогают отличить ошибку транспорта от отсутствия перерисовки интерфейса.

Симптом	Вероятная причина	Что проверить
Нет пункта USB-экран	Возможность выключена	`--show-config`, затем собрать с `MK61_ENABLE_USB_SCREEN=1`
Список портов пуст	Кабель без data, нет драйвера или прав	Другой кабель; `flutter doctor`; права `/dev/ttyACM*/ttyUSB*`
«Ожидание USB Screen»	Режим был явно завершён на MK61 либо firmware старая	Нажать «Включить USB Screen»; проверить `MK61_ENABLE_USB_SCREEN=1` и версию firmware
Порт занят	Его открыл другой terminal	Закрыть Serial Monitor, `screen`, `minicom` и второй экземпляр приложения
Физический экран не гаснет	Handshake не дошёл до CAPS	Статус приложения, совместимость wire version и ошибки parser
Связь теряется в FOCAL/TinyBASIC	Старая firmware не обслуживает idle внутри редактора	Собрать текущую firmware и повторить тест более 3 секунд
Повреждён или неполон кадр	CRC/sequence error	Клиент сам запросит keyframe; проверить кабель и одинаковую version 2

Симптом	Вероятная причина	Что проверить
Буквы зависят от русской раскладки	Старый клиент либо открыт terminal input	Собрать текущий клиент; скрыть терминальную шторку
Повторная буква даёт неверный символ	Ввод идёт legacy down/up вместо terminal `kbd`	Проверить capability `TERMINAL_KBD` и версии firmware/клиента
После выхода виден калькулятор вместо FOCAL	Старая логика без foreground restore	Обновить firmware; проверить display revision в редакторе
Linux: permission denied	Пользователь не состоит в группе порта	Добавить в `dialout`/нужную группу и войти заново
Клиент аварийно закрыт	`DETACH` не отправлен	Подождать до 3 секунд либо удерживать физический ESC 1,5 секунды

Если изменённые USB descriptors не прошли безопасный allowlist, выключите «Автоподключение», нажмите «Обновить», выберите порт вручную и подключитесь. На macOS используйте `/dev/cu.usbmodem...`, а не Bluetooth-порт. Для проверки firmware вне приложения можно временно открыть обычный terminal, выполнить `ver`, затем закрыть его до запуска MK61 USB Screen.

Совместимость и развитие протокола

- Version 2 проверяется строго; пакет другой версии отклоняется до обработки.
- Неизвестный тип сообщения можно игнорировать, но новый обязательный feature должен иметь capability bit либо новую protocol version.
- `CAP_DIFF` и `plan_diff()` зарезервированы. Desktop FrameAssembler уже умеет начать инкрементальный кадр с копии опубликованного framebuffer при `keyframe=0`; текущая firmware намеренно всегда посылает восемь полных страниц.
- Новый codec требует нового codec id и механизма объявления поддержки. Старый клиент корректно отклонит неизвестный codec и запросит keyframe.
- Геометрия 192x64 и page height 8 проверяются клиентом. Их изменение не является прозрачным расширением и требует согласованного обновления клиента.
- Максимальный payload 224 байта и максимальная рамка 238 байт являются частью реализации version 2.
- Текстовый канал не допускает NUL. Если будущему терминалу понадобится NUL, его следует оформить отдельным бинарным сообщением.

Протокол не имеет аутентификации или шифрования: он предназначен для локального USB-кабеля, а не для недоверенной сети. Встроенный терминал обладает теми же правами, что обычный USB terminal, включая команды управления устройством и переход в DFU. Подключать устройство следует только к доверенному host.

Надёжность

- USB-передача не может бесконечно блокировать калькулятор.
- Промежуточные ревизии экрана объединяются, но передаваемый snapshot неизменяем.
- Все пакеты имеют версию, ограниченную длину и sequence.
- Клиент публикует кадр атомарно после `FRAME_END`.
- Нарушение sequence всегда приводит к запросу keyframe.
- Текст терминала никогда не вставляется внутрь бинарной рамки.
- Heartbeat возвращает физический дисплей без участия desktop-приложения.
- Потеря фокуса и отключение снимают все виртуально удерживаемые клавиши.

- Вход в режим не выделяет heap и не зависит от успешности `malloc`.
- Переключение backend не назначает калькулятор владельцем foreground.

Автоматическая проверка

Из корня проекта выполните чистые host-тесты протокола и поверхности:

```
./tests/run_usb_screen_protocol_tests.sh
./tests/run_usb_screen_surface_tests.sh
```

Полный набор firmware host-тестов:

```
./tests/run_all_tests.sh
```

Для desktop-клиента:

```
cd apps/mk61_usb_screen
flutter analyze
flutter test
```

Набор Flutter-тестов включает COBS/CRC/PackBits, atomic FrameAssembler, capabilities, три раскладки, fixed US-QWERTY, widget layout и сквозную fake serial-сессию `uscreen -> OFFER -> ATTACH -> CAPS -> frame`, terminal multiplex, клавиши, heartbeat, sequence/CRC recovery, подавление автозапуска после device-side `DETACH` и явное повторное включение.

Минимальная compile-проверка firmware должна включать как минимум A00 с `MK61_ENABLE_USB_SCREEN=1`; release workflow делает именно такую сборку. Перед аппаратным выпуском рекомендуется дополнительно собрать выбранные A02/UC1609 профили и сравнить размер с лимитами STM32F411.

PDF этого руководства собирается тем же проектным генератором:

```
python3 doc/build_md_pdf.py \
  doc/src/MK61s-mini-USB-Screen.md
```

Результат находится в `doc/MK61s-mini-USB-Screen.pdf`.

Критерии готовности

- Сборки LCD1602 A00, LCD1602 A02, OLED1602 WS0010 и UC1609 с `MK61_ENABLE_USB_SCREEN=1` входят в USB-экран без отдельной специальной версии для конкретного дисплея.
- При `MK61_ENABLE_USB_SCREEN=0` код режима, его статические буферы и пункт меню отсутствуют в прошивке.
- В USB-режиме они дают одинаковую виртуальную графическую поверхность.
- В LCD1602-сборке пункт шрифта виден только на активной виртуальной графической поверхности; пресет применяется сразу и сохраняется для следующей USB-сессии.
- После handshake физический экран погашен.
- После запуска desktop-клиента USB Screen автоматически включается из текущего foreground-интерфейса без выбора пункта меню.
- В клиенте без расхождений видны меню, редакторы, специальные символы, курсор, WBMP, Markdown, CHIP-8 и шрифты.
- В LCD1602-сборке WBMP и CHIP-8 компилируются только вместе с USB Screen, запускаются только после `ATTACH` и завершаются с сообщением ГРАФИКА / НЕДОСТУПНА при потере графического backend.
- Markdown доступен без USB Screen; при `ATTACH` он использует графическое оформление и WBMP-блоки, а после разрыва возвращается к plain text.
- Приложение масштабирует 192x64 на любое окно без сглаживания пикселей.
- Обрыв связи не зависает прошивку и возвращает физический экран.

- Кодек, parser и terminal multiplex покрыты host-тестами, включая фрагментацию, повреждённые пакеты и текст между кадрами.
- Desktop protocol, atomic frame assembler и три раскладки клавиатуры покрыты Flutter-тестами; release-приложение собирается с вложенным `libserialport`.
- Workflow `.github/workflows/usb-screen-desktop.yml` выполняет анализ и тесты, затем нативно собирает и упаковывает macOS, Windows x64 и Linux x64.

Аппаратный smoke-test

Для окончательной проверки конкретной платы:

1. прошить соответствующий файл из `.build/firmware/`;
2. оставить устройство на обычном экране калькулятора, подключить USB и запустить desktop-клиент; убедиться, что USB Screen включился автоматически;
3. убедиться, что до ATTACH физический экран остаётся включённым, а после появления изображения в приложении полностью гаснет;
4. открыть калькулятор, меню, проводник, создать новый файл FOCAL и оставить редактор без ввода дольше 3 секунд; проверить, что связь не теряется, а `New file`/`New folder` либо Новый файл/Новая папка соответствуют языку устройства;
5. проверить многострочный вид, кириллицу, курсор, часы, WBMP и виртуальные клавиши с удержанием; запустить программы MK-61, FOCAL, TinyBASIC и совместимый `.ch8`, убедиться, что экран и одновременный ввод продолжают работать;
6. выполнить во встроенном терминале `ver`, `help` и `reg` во время обновления экрана, убедиться в целых кадрах и полном выводе команд;
7. поочерёдно закрыть приложение из калькулятора, меню и редактора FOCAL/TinyBASIC; проверить не позднее heartbeat timeout включение физического экрана с текущим интерфейсом и без промежуточного экрана калькулятора;
8. повторить автозапуск из редактора FOCAL без выхода из него; затем выполнить аварийный выход долгим физическим ESC, убедиться, что клиент не включает режим обратно, и проверить кнопку «Включить USB Screen».
9. запустить WBMP и CHIP-8 на A00/A02, закрыть desktop-клиент и проверить, что каждый модуль завершился, физический LCD включился и показал ГРАФИКА / НЕДОСТУПНА.
10. открыть Markdown на A00/A02, подключить и отключить desktop-клиент и проверить переход между графическим документом и plain text без выхода из просмотрщика.

Необязательный родной кадр PicoCalc 320×320

PicoCalc 0.8.0 объявляет `CAP_NATIVE_320 = 1 << 8` в десятибайтовом `OFFER`, сохраняющем геометрию 192×64 для старых клиентов. Новый клиент отвечает `ATTACH` с одним байтом `01`. Только тогда `CAPS` имеет 16 байт: ширина в исходном поле `u16` равна 320, в конце добавлено `native_height:u16 = 320`. Исходное поле `height:u8` содержит младший байт (64). Без такого ATTACH сохраняется прежний CAPS из 14 байт и прежний поток 192×64.

FRAME_BEGIN содержит 80 прямоугольников. Каждый из 40 восьмипиксельных вертикальных страниц передаётся двумя RECT шириной 160, с x=0 и x=160. Это сохраняет шестибайтовый заголовок RECT, однобайтовые x/page/width и предел payload 224 байта. FRAME_END проверяет CRC16 всех 12800 байт. Снимок LCD остаётся неизменным до FRAME_END; следующий кадр снимается заново. Клиент публикует только полностью собранные кадры. Handshake STM32 не меняется.